

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО”



Технології графічного процесінгу

Data Parallel Computing

Семінар

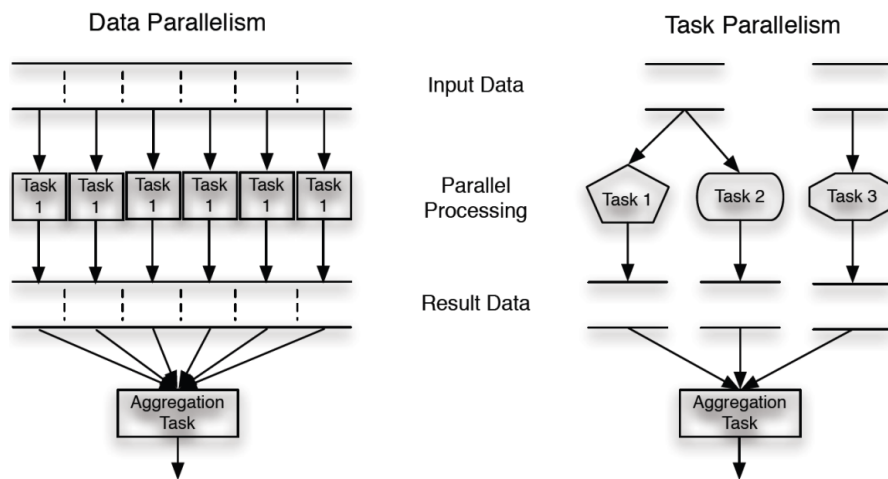
Виконав:
Олександр Ковальов
Група:
ІМ-51мн
Курс:
1

2 травня 2026 р.

1 Вступ та поняття паралелізму даних

Паралелізм даних є фундаментальною парадигмою в сучасних обчисленнях, яка дозволяє досягти високої продуктивності шляхом одночасного виконання однакових операцій над великими масивами даних. У контексті архітектури графічних процесорів цей підхід стає ключовим, оскільки він дозволяє ефективно масштабувати алгоритми залежно від кількості доступних обчислювальних ядер. Основна ідея полягає в тому, що програма розбивається на безліч ідентичних незалежних потоків, кожен з яких оперує своєю унікальною частиною загального набору інформації. Такий метод обробки дозволяє перетворити лінійну складність виконання на константну або логарифмічну в ідеальних умовах, що є критично важливим для наукових розрахунків, криптографії та обробки сигналів.

Для глибокого розуміння предмета необхідно чітко розрізняти паралелізм задач та паралелізм даних. Паралелізм задач фокусується на одночасному виконанні різних функцій або ділянок коду, які можуть працювати як з тими самими, так і з різними даними. Цей тип паралелізму зазвичай реалізується на багатоядерних центральних процесорах, де кожне ядро виконує окрему логічну гілку програми. Натомість паралелізм даних, який лежить в основі моделі CUDA, зосереджений на розподілі елементів масивів між тисячами потоків для виконання однієї і тієї самої інструкції. Якщо паралелізм задач обмежений кількістю функціональних блоків у програмі, то паралелізм даних масштабується природним чином разом із розміром вхідних даних, що робить його набагато ефективнішим для масивних обчислень [1].

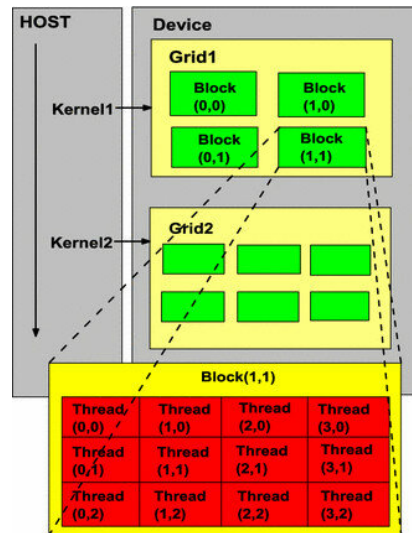


Графічні процесори (GPU) є ідеальним середовищем для реалізації паралелізму даних через їхню архітектурну специфіку, орієнтовану на високу пропускну здатність (throughput). На відміну від центральних процесорів, які спроектовані для мінімізації затримки (latency) виконання одного потоку за допомогою складних систем передбачення переходів та багаторівневого кешування, GPU складаються з величезної кількості спрощених ядер. Така структура дозволяє одночасно підтримувати виконання тисяч потоків, де затримки доступу до пам'яті приховуються за рахунок масовості обчислень. Саме ця здатність GPU до швидкої паралельної обробки робить їх незамінними в технологіях графічного процесингу та загальних обчисленнях на GPU (GPGPU), де домінує модель програмування CUDA [2].

2 Архітектура гетерогенних обчислень CUDA

Архітектура CUDA базується на гетерогенній моделі обчислень, яка ефективно об'єднує центральний процесор, що виступає в ролі хоста, та графічний процесор, який виконує роль пристрою. У цій системі кожен тип мікропроцесора виконує ті класи завдань, для яких його апаратна структура підходить найкраще. Хост відповідає за виконання складних послідовних ділянок коду, керування загальною логікою потоку програми, операції вводу-виводу та взаємодію з операційною системою. Пристрій, у свою чергу,

функціонує як потужний математичний співпроцесор, який активується виключно для масивних паралельних обчислень. Критично важливою особливістю цієї моделі є те, що хост і пристрій мають фізично ізольовані простори пам'яті, що вимагає від програміста явного керування переміщенням інформації між оперативною пам'яттю системи та відеопам'яттю графічного прискорювача.



Типова програма, написана з використанням розширення CUDA C, структурно поєднує в одному сирцевому файлі як послідовний код хоста, так і паралельний код пристрою. Для обробки такого коду використовується спеціалізований інструментарій компілятора nvcc від NVIDIA, який на етапі трансляції автоматично розділяє ці два типи коду. Інструкції, призначені для центрального процесора, передаються стандартному компілятору хост-системи, тоді як паралельні функції, які називаються ядрами, транслуються у проміжний код PTX або безпосередньо в бінарний код для архітектури GPU. Виконання будь-якого гетерогенного застосунку завжди починається на центральному процесорі, який контролює весь подальший життєвий цикл програми та ініціює обчислення на графічному прискорювачі.

Процес взаємодії між хостом і пристроєм у межах типової CUDA-програми складається з кількох чітко визначених фаз виконання, що йдуть одна за одною. На початковому етапі центральний процесор за допомогою спеціальних функцій API виділяє необхідний об'єм простору в глобальній пам'яті графічного процесора та ініціює копіювання масивів вхідних даних зі своєї оперативної пам'яті до пам'яті пристрою. Після успішного завершення передачі даних хост виконує запуск ядра, вказуючи конфігурацію виконання, яка визначає загальну кількість паралельних потоків для одночасної обробки інформації. Під час роботи графічного процесора центральний процесор може або виконувати інші незалежні інструкції, або синхронізуватися та очікувати завершення паралельних обчислень. На завершальній фазі хост копіює отримані результати з пам'яті пристрою назад до своєї оперативної пам'яті та звільняє раніше виділені ресурси відеопам'яті, що дозволяє уникнути витоків пам'яті та коректно завершити роботу алгоритму.

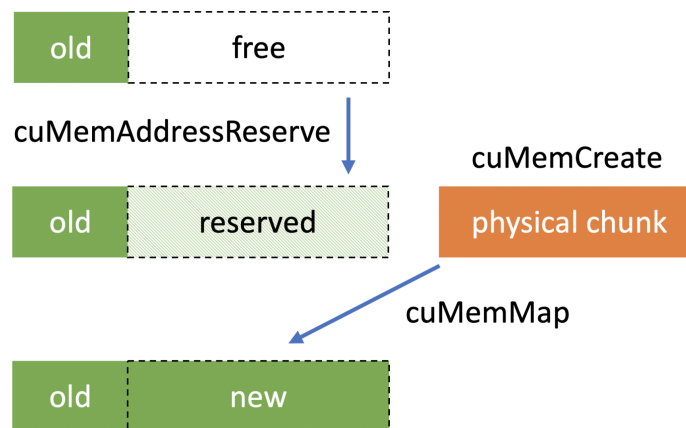
3 Керування пам'яттю пристрою

Керування глобальною пам'яттю пристрою є критично важливим аспектом програмування на CUDA, оскільки центральний та графічний процесори мають фізично відокремлені простори пам'яті. Робота з даними на графічному прискорювачі передбачає суворий контроль за їхнім життєвим циклом, який ініціюється та керується виключно кодом хоста. Глобальна пам'ять пристрою відіграє ключову роль у гетерогенних обчисленнях, адже вона є найбільшим доступним сховищем, до якого мають прямий доступ усі потоки під час виконання ядра. Для маніпуляцій з цим адресним простором CUDA Runtime API надає спеціалізовані функції, які є концептуальними аналогами стандартних засобів керування пам'яттю в мові C, але суворо адаптовані для ізольованого середовища GPU.

Першим етапом у життєвому циклі даних є виділення необхідного об'єму відеопам'яті за допомогою функції `cudaMalloc`. Ця функція вимагає передачі вказівника на вказівник, за яким буде записано базову адресу виділеної області пам'яті пристрою, а також загального розміру необхідного блоку в байтах. Розмір традиційно обчислюється шляхом множення кількості елементів масиву на розмір одного елемента, визначений за допомогою оператора `sizeof`. Вкрай важливо розуміти, що адреси, які повертає `cudaMalloc`, є дійсними виключно в межах адресного простору графічного пристрою. Будь-яка спроба прямого доступу або розмінування цих вказівників у кодї, що виконується на центральному процесорі, неминуче призведе до критичної помилки доступу до пам'яті (segmentation fault) та аварійного завершення програми [1].

Після успішного резервування відеопам'яті виникає необхідність перенесення вхідних масивів інформації з оперативної пам'яті системи до глобальної пам'яті графічного прискорювача. Цю задачу вирішує функція `cudaMemcpy`, яка забезпечує надійне копіювання блоків даних між різними апаратними вузлами. Окрім вказівників на цільову та вихідну області, а також об'єму даних у байтах, ця функція приймає спеціальну системну константу, що чітко вказує напрямком транзакції. Для завантаження даних на пристрій перед ініціалізацією обчислень застосовується прапорець `cudaMemcpyHostToDevice`, тоді як для повернення розрахованих результатів назад на хост використовується напрямок `cudaMemcpyDeviceToHost`. Ця операція передачі є синхронною для центрального процесора, що змушує хост призупинити подальше виконання інструкцій до моменту повного завершення процесу копіювання по шині PCIe.

Завершальним і обов'язковим етапом роботи є коректне звільнення раніше виділених апаратних ресурсів після того, як необхідність у них відпадає. Для цього застосовується функція `cudaFree`, яка приймає вказівник на область пам'яті пристрою та повертає її до загального пулу доступної відеопам'яті. Ігнорування цієї операції призводить до серйозних витоків пам'яті на графічному прискорювачі. Оскільки ресурси GPU часто є спільними для кількох процесів або навіть для графічної оболонки операційної системи, вичерпання глобальної пам'яті може спричинити не лише збої у поточній програмі, але й деградацію продуктивності всієї системи. Таким чином, дисципліноване керування життєвим циклом даних є основою для створення стабільних та ефективних паралельних застосунків.



4 Написання ядра (Kernel) на прикладі додавання векторів

Ядро (kernel) у термінології CUDA – це спеціальна C-функція, яка викликається з коду хоста, але виконується виключно на апаратних ресурсах графічного пристрою. На відміну від звичайних функцій центрального процесора, ядро виконується паралельно величезною кількістю незалежних потоків. Для того, щоб компілятор nvcc міг коректно розпізнати та відокремити код пристрою від коду хоста, застосовуються спеціальні специфікатори простору виконання. Ключовим із них є специфікатор `__global__`,

який сигналізує компілятору про те, що функція є точкою входу для паралельного виконання на GPU. Функції, задекларовані з цим специфікатором, обов'язково повинні мати тип повернення `void`, оскільки їхній виклик з боку хоста є асинхронним оператором, який не передбачає прямого повернення результату в викликаючий потік центрального процесора.

Класичним прикладом для демонстрації переходу від послідовної моделі виконання до паралельної є операція покомпонентного додавання двох векторів. У традиційному програмуванні для виконання цієї задачі використовується цикл, який ітерує через усі елементи масивів, послідовно обробляючи кожну пару значень. У паралельній моделі програмування CUDA необхідність у явному циклі зникає. Замість ітеративної обробки генерується сітка потоків, де кожен окремий потік бере на себе відповідальність за обчислення лише одного елемента вихідного масиву. Таким чином, код паралельного ядра описує математичну логіку виключно для однієї скалярної операції, а необхідна масовість обчислень забезпечується архітектурою GPU під час ініціалізації запуску цього ядра.

Основою будь-якого ядра CUDA є визначення потоком своєї глобальної ідентичності, щоб він міг звернутися до правильної комірки в пам'яті. Потік розраховує свій унікальний глобальний індекс за допомогою вбудованих апаратних змінних, після чого виконує обов'язкову перевірку виходу за межі масиву. Ця перевірка є критично важливою, оскільки загальна кількість апаратно запущених потоків найчастіше округлюється до розміру блоку і може перевищувати фактичний розмір корисних даних. Тільки у випадку успішного проходження перевірки потік виконує читання операндів з глобальної пам'яті, їх додавання та запис кінцевого результату. Нижче наведено базову реалізацію такого ядра:

```
// CUDA kernel for vector addition
__global__
void vecAddKernel(float* A, float* B, float* C, int n) {
    // Calculate global thread index based on block and thread IDs
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    // Check boundary conditions to prevent memory access violations
    if (i < n) {
        C[i] = A[i] + B[i];
    }
}
```

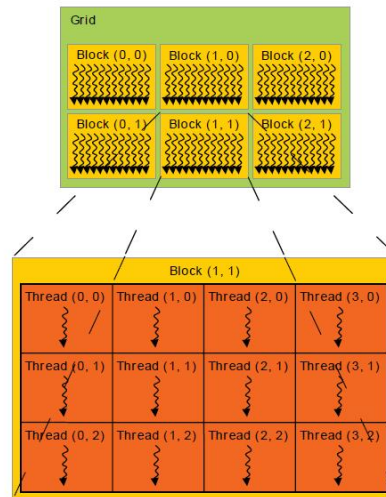
5 Організація потоків (Grids, Blocks, Threads)

Ключовою концепцією, що забезпечує масштабованість паралельних програм у середовищі CUDA, є дворівнева ієрархія організації потоків під час виконання ядра. Коли центральний процесор ініціює запуск паралельної функції на графічному прискорювачі, він створює загальну сітку (grid), яка складається з великої кількості незалежних потоків. Для забезпечення ефективного керування апаратними ресурсами та гнучкості планування ця загальна сітка логічно поділяється на менші структурні одиниці, які називаються блоками (blocks). Кожен блок об'єднує певну кількість потоків, які виконуються на одному потоковому мультипроцесорі (Streaming Multiprocessor). Розмірність сітки та блоків може бути одновимірною, двовимірною або тривимірною, що дозволяє програмістам природним чином відображати абстрактну багатовимірну топологію даних, таку як матриці або об'ємні зображення, на фізичну архітектуру виконання.

Для визначення своєї унікальної позиції в цій ієрархії кожен потік отримує доступ до спеціальних вбудованих регістрів апаратного забезпечення, які ініціалізуються системою автоматично під час запуску ядра. Змінна `threadIdx` визначає локальний індекс потоку всередині його рідного блоку, тоді як змінна `blockIdx` вказує на координати самого блоку в межах загальної сітки. Крім того, потокам доступні змінні `blockDim` та `gridDim`, які зберігають інформацію про конфігурацію розмірності блоку та сітки відповідно. Усі ці змінні мають спеціальний векторний тип, що містить поля `x`, `y` та `z`. У найпростішому випадку обробки одновимірних масивів, як у прикладі з додаванням векторів, використовується лише координата `x`, що значно спрощує математику обчислення індексів та робить код більш читабельним.

Для правильної індексації масивів даних кожен потік повинен трансформувати свої ієрархічні координати в єдиний глобальний лінійний індекс. Ця операція є критичною для концепції паралелізму

даних, оскільки вона гарантує, що кожен потік звертатиметься до своєї унікальної комірки пам'яті без конфліктів доступу. Глобальний індекс обчислюється за фундаментальною формулою: індекс блоку (blockIdx.x) множиться на розмір блоку (blockDim.x), і до отриманого базового зсуву додається локальний індекс потоку (threadIdx.x). Це можна уявити як поділ довгого масиву на рівні сегменти: спочатку розраховується адреса початку конкретного сегмента, призначеного для обробки певним блоком, а потім кожен потік цього блоку робить необхідний відступ для доступу до свого елемента.



Після обчислення глобального індексу абсолютною необхідністю є застосування перевірки граничних умов. Специфіка конфігурації запуску ядра CUDA вимагає, щоб загальна кількість потоків у сітці була кратною розміру блоку. Оскільки розмір масиву вхідних даних рідко ідеально збігається з цим кратним значенням, під час виконання зазвичай створюється дещо більше потоків, ніж існує фактичних елементів для обробки. Ті «зайві» потоки, глобальний індекс яких перевищує або дорівнює загальній кількості елементів у масиві, повинні бути програмно ізольовані від виконання операцій з пам'яттю. Тільки за умови суворого дотримання цієї перевірки програма гарантовано уникне виходу за межі виділеної пам'яті та працюватиме стабільно на будь-яких об'ємах даних.

6 Перспективи розвитку та альтернативні абстракції

Незважаючи на те, що базова модель CUDA C надає розробнику максимальний контроль над апаратними ресурсами графічного процесора, вона має суттєві недоліки з точки зору ергономіки розробки та безпеки програмного забезпечення. Необхідність ручного виділення та звільнення пам'яті через вказівники, явне обчислення глобальних індексів потоків та самостійний контроль за межами масивів створюють сприятливе середовище для помилок доступу до пам'яті та станів гонитви. Сучасні тенденції в інженерії програмного забезпечення демонструють запит на більш безпечні та декларативні абстракції для паралельних обчислень. Яскравим прикладом такого підходу є концепція паралельних ітераторів, де розробник описує трансформацію даних на високому рівні, а бібліотека автоматично розподіляє навантаження [3]. Хоча класичні реалізації паралельних ітераторів орієнтовані на багатоядерні центральні процесори, сама ідея приховування низькорівневої ієрархії потоків за безпечним інтерфейсом є надзвичайно перспективною для GPU.

У цьому контексті використання сучасних системних мов програмування, таких як Rust, замість класичного C++, вбачається логічним еволюційним кроком для екосистеми гетерогенних обчислень. Використання Rust дозволяє перенести гарантії безпеки доступу до пам'яті на етап компіляції. Завдяки системі володіння (ownership) та концепції життєвого циклу (lifetimes) можна створити безпечні обгортки навколо API CUDA, де відеопам'ять буде автоматично вивільнятися при виході з області видимості, унеможливаючи витрати ресурсів. Більше того, проекти на кшталт rust-gru вже зараз дозволяють компілювати безпечний код безпосередньо у проміжне представлення PTX для виконання на відео-

картах [4]. Перехід до таких технологій дозволяє поєднати безпрецедентну обчислювальну потужність графічних процесорів із сучасними парадигмами безпечної розробки, знижуючи поріг входження та вартість підтримки складних паралельних систем.

7 Висновки

Підсумовуючи розглянутий матеріал, можна стверджувати, що архітектура CUDA заклала фундаментальні принципи для практичної реалізації Data Parallel Computing на графічних процесорах. Розподіл відповідальності між хостом та пристроєм, чіткий життєвий цикл керування глобальною відеопам'яттю та масштабована ієрархія організації потоків у вигляді сіток і блоків забезпечують високу пропускну здатність для масивних розрахунків. Розуміння цих базових низькорівневих механізмів є абсолютно необхідним для ефективної оптимізації продуктивності та розуміння того, як програмний код взаємодіє з апаратним забезпеченням на фізичному рівні. Без цієї бази неможливе подальше глибоке занурення в технології графічного процесингу.

Водночас розвиток індустрії вказує на те, що майбутнє паралельних обчислень лежить у площині поєднання цієї низькорівневої потужності з безпечними абстракціями. Базова модель CUDA C вимагає від програміста надмірної уваги до рутинних операцій, що часто призводить до критичних помилок. Впровадження сучасних системних інструментів та декларативних підходів відкриває шлях до створення надійнішого програмного забезпечення. Еволюція від ручного розрахунку індексів до безпечних абстракцій над даними дозволить розробникам фокусуватися на математичній логіці алгоритмів, залишаючи системі завдання безпечного та ефективного розподілу обчислень по тисячах ядер графічного прискорювача.

Література

- [1] D. B. Kirk and W.-m. W. Hwu, *Programming massively parallel processors: a hands-on approach*, 3rd ed. Morgan Kaufmann, 2016.
- [2] *CUDA C++ Programming Guide*, NVIDIA Corporation, 2023. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- [3] N. Matsakis *et al.*, “Rayon: A data-parallelism library for rust,” <https://github.com/rayon-rs/rayon>, 2023.
- [4] Embark Studios, “Rust-gpu: Towards making rust a first-class language and ecosystem for gpu code,” <https://github.com/EmbarkStudios/rust-gpu>, 2023.